

Iterated Sums and Products from One

Correctness and Quantitative Growth of OEIS A352969

A research note prepared for Vladimir Reshetnikov

September 20, 2026

Abstract

Let $S_0 = \{1\}$ and let S_{n+1} consist of all sums and products of two members of S_n , allowing equal operands. We prove that the Python program linked from OEIS A352969 computes S_n exactly, under its natural input and cache-integrity assumptions. We then establish unconditional quantitative growth estimates. Writing $a_n = |S_n|$, we prove

$$\log_2 \log_2 a_n = n + O(\sqrt{\log n}), \quad \frac{\log a_{n+1}}{\log a_n} \longrightarrow 2.$$

The first estimate is made explicit: for $n \geq 16$, if $t = \lceil \log_2(2n) \rceil$ and r is the smallest integer with $t \leq r(r+1)/2$, then $2^{2^{n-r-4}} \leq a_n \leq 2^{2^{n-1}}$. The proof combines a low-depth radix construction for arbitrary positive integers with collision-free products of prime multisets. An elementary divisor bound then controls the loss in one multiplication step. The normalized logarithms $2^{-n} \log_2 a_n$ decrease to a limit, but its value and strict positivity are not determined here. Thus the result is a sharp logarithmic-scale growth theorem, not a claimed multiplicative asymptotic equivalent for a_n . Reproducible code, independently checked small-stage data, and an executable expression certificate accompany the article.

Contents

1	The sequence and the main results	3
2	What the Python program does	4
2.1	Why unordered pairs are sufficient	5
2.2	Correctness and termination	5
3	Structural facts about the generated sets	6
3.1	Persistence and arithmetic-expression height	6
3.2	Maximum value and powers of two	6
3.3	Cardinality bounds and the entropy limit	7
4	Why a lower bound needs a construction	7
5	An elementary supply of primes	8
6	A first complete growth proof	9
7	A sharper construction for arbitrary integers	9
7.1	What the construction proves about individual integers	10
7.2	Executable realization	11
8	Proof of the explicit counting bound	11
9	One-step growth through a divisor bound	12
10	The precise computational cost	13
10.1	Candidate counts and unit-cost arithmetic	13
10.2	Bit operations and memory	14
10.3	An output-sensitive consequence	14
11	Recomputed data and verification	14
12	What is established, and what remains to refine	15
A	Contents and reproduction instructions	16
B	Source provenance	17

1 The sequence and the main results

For a finite set A of positive integers, write

$$A + A = \{x + y : x, y \in A\}, \quad AA = \{xy : x, y \in A\}, \quad \Phi(A) = (A + A) \cup AA.$$

The sequence under consideration is

$$S_0 = \{1\}, \quad S_{n+1} = \Phi(S_n), \quad a_n = |S_n|. \quad (1)$$

This is OEIS A352969, entered by Vladimir Reshetnikov in April 2022. The entry attributes its Python program to Chai Wah Wu and currently lists

$$1, 2, 4, 11, 52, 678, 67144, 357306081 \quad (2)$$

for $n = 0, \dots, 7$ [1]. We independently recomputed the first seven of these values, through $n = 6$; the value at $n = 7$ is used only as reported OEIS data. All logarithms denoted by $\log$ are natural, and $\log_2$ denotes base two.

Two distinct tasks must be separated. Proving the program correct amounts to checking that its enumeration implements (1) without omissions or spurious values. Estimating a_n is harder: the program counts *values*, not arithmetic expressions, and many expressions have the same value.

Theorem 1.1 (Quantitative growth). *For $n \geq 16$, define*

$$t_n = \lceil \log_2(2n) \rceil, \quad r_n = \min\{r \in \mathbb{N} : t_n \leq r(r+1)/2\}.$$

Then

$$\boxed{2^{2^{n-r_n-4}} \leq a_n \leq 2^{2^{n-1}}}. \quad (3)$$

In particular,

$$n - \sqrt{2 \log_2 n} - O(1) \leq \log_2 \log_2 a_n \leq n - 1, \quad (4)$$

$$\lim_{n \rightarrow \infty} \frac{\log \log a_n}{n} = \log 2, \quad (5)$$

$$\log a_n = \frac{2^n}{n^{o(1)}}. \quad (6)$$

Moreover, there is a nonnegative sequence ε_n such that

$$a_{n+1} = a_n^{2-\varepsilon_n}, \quad 0 \leq \varepsilon_n \leq n^{-1+o(1)}. \quad (7)$$

Consequently $\log a_{n+1} / \log a_n \rightarrow 2$.

The notation in (6) means that the positive factor $2^n / \log a_n$ grows more slowly than every fixed positive power of n ; it is not an assertion that this factor tends to infinity. An equivalent and less compressed statement is that for every $\delta > 0$ and all sufficiently large n ,

$$\frac{2^n}{n^\delta} \leq \log a_n \leq \frac{\log 2}{2} 2^n.$$

The threshold in the lower bound may depend on δ .

Theorem 1.2 (Normalized logarithms). *The sequence*

$$\beta_n = 2^{-n} \log_2 a_n \quad (n \geq 1)$$

is nonincreasing and has a limit $\theta \geq 0$. Thus

$$a_n = 2^{(\theta+o(1))2^n}. \quad (8)$$

The independently recomputed data imply $\theta \leq 0.2505464198$; the reported OEIS value of a_7 improves this to $\theta \leq 0.2219733221$.

Formula (8) does not by itself prove $\theta > 0$. Even a positive value of θ would give an equivalent for $\log a_n$, not automatically a multiplicative equivalent $a_n \sim 2^{\theta 2^n}$. These distinctions matter for a doubly exponential sequence.

2 What the Python program does

The OEIS listing defines a cached function named `A352969_set`. Its base case returns `{1}`. At a positive argument it forms two sets: one from the sums, the other from the products, of `combinations_with_replacement(previous, 2)`, and returns their union. A final wrapper takes the cardinality. In the retrieved listing that wrapper is named `A353969`; this is a naming typo, not a mathematical change [1].

The following independently written version expresses the same recurrence in one pass over the pairs. It deliberately returns immutable results and validates the public input before entering the cache.

```
from functools import lru_cache
from itertools import combinations_with_replacement

@lru_cache(maxsize=None)
def _stage(n):
    if n == 0:
        return frozenset((1,))
    result = set()
    for x, y in combinations_with_replacement(_stage(n - 1), 2):
        result.add(x + y)
        result.add(x * y)
    return frozenset(result)

def reachable(n):
    if isinstance(n, bool) or not isinstance(n, int):
        raise TypeError("n must be an integer")
    if n < 0:
        raise ValueError("n must be nonnegative")
    return _stage(n)

def A352969(n):
    return len(reachable(n))
```

The package contains both cached and iterative variants, with additional input checks and an optional pair-count budget. The compact listing above is for exposition; the executable implementation is `code/a352969.py`.

2.1 Why unordered pairs are sufficient

Fix an arbitrary enumeration $A = \{u_1, \dots, u_m\}$. The iterator with replacement and length two visits exactly the index pairs

$$(i, j) \quad \text{with} \quad 1 \leq i \leq j \leq m.$$

Its ordering is determined by the input iterable, not by the numerical order of its elements [2]. A Python set therefore need not be sorted before it is passed to the iterator.

For every ordered pair $(x, y) \in A^2$, exactly one of (x, y) and (y, x) occurs in this unordered enumeration, except that (x, x) occurs once. Both operations commute, so replacing an ordered pair by its reversal changes neither output. Equal pairs must be retained: for example, the first step needs $1 + 1$ and $1 \cdot 1$.

Lemma 2.1 (One-step enumeration). *For any finite set A of positive integers, the union of the sum outputs and product outputs from the unordered enumeration with replacement is exactly $\Phi(A)$.*

Proof. Every emitted sum or product has both operands in A , so no emitted value lies outside $\Phi(A)$. Conversely, if $z = x + y$ or $z = xy$ with $x, y \in A$, the iterator visits the unordered pair containing x and y . Commutativity ensures that the corresponding output equals z . Set insertion discards repeated values but never discards their membership. Taking the union retains values produced by either operation. $\square$

2.2 Correctness and termination

Theorem 2.2 (Correctness of the OEIS algorithm). *For every nonnegative integer n , the OEIS set function returns S_n , provided its previously cached sets have not been externally modified and the execution has sufficient memory and stack capacity. Its cardinality wrapper therefore returns a_n .*

Proof. Induct on n . At $n = 0$, the returned set is $\{1\} = S_0$. For $n > 0$, the induction hypothesis identifies the recursively returned set with S_{n-1} . Lemma 2.1 identifies the two comprehensions and their union with $\Phi(S_{n-1}) = S_n$. The wrapper applies cardinality to this exact set.

For mathematical termination, the recursion decreases the nonnegative integer argument by one. At each level the previous set is finite, and there are only $m(m+1)/2$ unordered pairs. Thus each loop is finite. Python's integer operations are exact on integer operands; there is no fixed-width overflow that silently changes the values [4, 5]. Resource exceptions are a separate issue from the mathematical termination of this finite procedure. $\square$

The cache changes reuse, not the recurrence. In a single-threaded cold call, each stage is actually computed once; the second reference to the previous stage in the original two-comprehension implementation reuses its cached value. The cache retains references to returned objects rather than defensive copies [3]. This explains the qualification about external mutation: inserting a foreign integer into a cached mutable set invalidates the induction hypothesis for later calls. Returning a `frozenset` removes that particular failure mode [4].

The original listing is intended for $n \in \mathbb{N}$. A negative input never reaches the base case by decrementing, and a noninteger input is not part of its mathematical contract. The public validation in the supplied implementation makes this contract explicit. The distinction between booleans and integers is checked deliberately, since Python booleans are an integer subtype [4].

3 Structural facts about the generated sets

3.1 Persistence and arithmetic-expression height

Lemma 3.1. *For every n , $1 \in S_n$ and $S_n \subseteq S_{n+1}$.*

Proof. The first assertion follows from $1 \cdot 1 = 1$. If $x \in S_n$, then $x = x \cdot 1 \in S_{n+1}$. $\square$

A legal expression has leaves all equal to 1 and internal vertices labeled $+$ or $\times$, each with two children. Its height is zero for a leaf and one plus the maximum child height at an internal vertex. Define

$$h(m) = \min\{\text{ht}(E) : E \text{ is a legal expression with value } m\}.$$

Every positive integer has such an expression, for example a sum of ones. This h minimizes height without first minimizing the number of leaves. It is therefore not the ordinary integer complexity recorded in A005245, nor the leaf-optimal depth convention of A210659 [6, 7].

Proposition 3.2. *For every $n \geq 0$,*

$$S_n = \{m \geq 1 : h(m) \leq n\}. \quad (9)$$

For $m > 1$, $h(m)$ is also specified by the finite recurrence

$$h(m) = 1 + \min \begin{cases} \max(h(u), h(v)) & : u + v = m, \quad u, v \geq 1, \\ \max(h(u), h(v)) & : uv = m, \quad u, v \geq 2. \end{cases} \quad (10)$$

Proof. An expression of height at most $n + 1$ has root operands of height at most n , except for the leaf expression, whose value 1 persists by Lemma 3.1. Induction on n proves (9) in both directions. Alternatively, shorter expressions can be padded by multiplications by 1.

An optimal expression for $m > 1$ cannot have a root multiplication by 1: removing that root would strictly lower its height. Every remaining root has one of the proper decompositions in (10), whose operands are smaller than m . Optimizing over these roots proves the recurrence. $\square$

The program for (10) supplies a useful independent test oracle: it indexes the computation by the numerical value m , whereas the OEIS algorithm indexes it by the stage n .

3.2 Maximum value and powers of two

Proposition 3.3. *Let $M_n = \max S_n$. Then*

$$M_0 = 1, \quad M_n = 2^{2^{n-1}} \quad (n \geq 1). \quad (11)$$

In particular $a_n \leq M_n$. For every integer $e \geq 1$,

$$h(2^e) = 1 + \lceil \log_2 e \rceil. \quad (12)$$

Proof. The largest sum and product from S_n are $2M_n$ and M_n^2 , respectively. Thus $M_{n+1} = \max(2M_n, M_n^2)$. The first step gives $M_1 = 2$; thereafter $M_n^2 \geq 2M_n$, giving (11) by repeated squaring. Every member is a positive integer at most M_n , so $a_n \leq M_n$.

For (12), form a balanced product of e copies of $2 = 1 + 1$. It has height at most $1 + \lceil \log_2 e \rceil$. Conversely $2^e \in S_j$ requires $2^e \leq 2^{2^{j-1}}$, hence $j \geq 1 + \lceil \log_2 e \rceil$. The two bounds coincide. $\square$

3.3 Cardinality bounds and the entropy limit

Proposition 3.4. *For all $n \geq 0$, $a_{n+1} \geq 2a_n$. For all $n \geq 1$,*

$$a_{n+1} \leq a_n^2. \quad (13)$$

Proof. Write the elements of $A = S_n$ increasingly as $1 = u_1 < u_2 < \cdots < u_m$. The sums

$$u_1 + u_1 < \cdots < u_1 + u_m < u_2 + u_m < \cdots < u_m + u_m$$

are $2m - 1$ distinct members of $A + A$. The additional value $1 \in AA$ is not in $A + A$, proving $|\Phi(A)| \geq 2m$.

For the upper bound, take $n \geq 1$, so $2 \in A$. Each of $A + A$ and AA has at most $m(m+1)/2$ members. Their intersection contains the m distinct values $2x$, $x \in A$: they occur both as $x + x$ and as $2 \cdot x$. Inclusion–exclusion gives

$$|\Phi(A)| \leq m(m+1) - m = m^2.$$

□

Dividing the logarithm of (13) by 2^{n+1} proves that $\beta_{n+1} \leq \beta_n$. Nonnegativity gives its limit θ , proving the existence assertion of Theorem 1.2. For every fixed $j \geq 1$,

$$a_n \leq a_j^{2^{n-j}} \quad (n \geq j). \quad (14)$$

The numerical upper bounds for θ follow by substituting the data at $j = 6$ or $j = 7$.

A useful consequence is that the sets are extremely sparse relative to the interval allowed by their largest element. Since $a_3 = 11$ and $M_3 = 16$, (14) and (11) give

$$\frac{a_n}{M_n} \leq \left(\frac{11}{16}\right)^{2^{n-3}} \longrightarrow 0 \quad (n \geq 3). \quad (15)$$

Thus having the same leading double-logarithmic scale as M_n does not mean that S_n has appreciable density in $[1, M_n]$.

4 Why a lower bound needs a construction

The raw number of candidate outputs at a stage is $a_n(a_n + 1)$. This is an upper bound, not a lower bound. Commutativity, associativity, distributivity, and numerical coincidences create many collisions. For instance $2 + 2 = 2 \cdot 2$, and $2 \cdot 6 = 3 \cdot 4$. Neither counting expression trees nor treating candidate values as independent random integers proves a lower bound for a_n .

We will instead use a family whose collisions are completely understood. The strategy has three parts: construct a long initial interval at a moderate height, select the primes in that interval, and multiply a prescribed number of them in a balanced tree. Unique factorization makes the last counting step exact. In particular, no assumption about the primality of specially structured expressions is needed: *every* integer in the chosen interval is constructible.

Lemma 4.1 (Prime-multiset amplification). *If S_k contains P distinct primes, then for every $n \geq k$, putting $R = 2^{n-k}$ gives*

$$a_n \geq \binom{P + R - 1}{R} \geq \left(\frac{P}{R}\right)^R. \quad (16)$$

Proof. Choose a multiset of exactly R primes from the P available primes, and multiply its elements in a complete balanced tree with R leaves. The multiplication tree adds exactly $n - k$ to the maximum input height, so its value belongs to S_n .

The prime multiplicities are recovered uniquely from the resulting integer. Hence different multisets give different values. The number of multisets is the binomial coefficient in (16). Finally,

$$\binom{P+R-1}{R} = \frac{P(P+1) \cdots (P+R-1)}{R!} \geq \frac{P^R}{R^R}.$$

□

The first inequality remains valid for any fixed family of pairwise coprime integers greater than one. Primes are convenient because an initial interval contains sufficiently many of them, as the next section proves without using the prime number theorem.

5 An elementary supply of primes

Lemma 5.1. *For every even integer $X \geq 16$,*

$$\pi(X) \geq \frac{X}{2 \log X}, \quad (17)$$

where $\pi(X)$ counts primes at most X .

Proof. Write $X = 2m$. The central binomial coefficient is the largest of the $2m + 1$ binomial coefficients in the expansion of $(1 + 1)^{2m}$. Therefore

$$\binom{2m}{m} \geq \frac{2^{2m}}{2m + 1} = \frac{2^X}{X + 1}.$$

For a prime p , counting multiples of each prime power in the factorials gives

$$v_p \binom{2m}{m} = \sum_{j \geq 1} \left(\left\lfloor \frac{2m}{p^j} \right\rfloor - 2 \left\lfloor \frac{m}{p^j} \right\rfloor \right).$$

Each summand is either zero or one and vanishes for $p^j > X$. Consequently $v_p \binom{2m}{m} \leq \left\lfloor \log_p X \right\rfloor$, so the full power of p dividing the binomial coefficient is at most X . All its prime factors are at most X , and hence

$$\binom{2m}{m} \leq X^{\pi(X)}.$$

Taking logarithms of both estimates yields

$$\pi(X) \geq \frac{X \log 2 - \log(X + 1)}{\log X}.$$

For $X \geq 16$, the numerator is at least $X/2$: this holds at 16, and $X(\log 2 - 1/2) - \log(X + 1)$ is increasing on $[16, \infty)$. This proves (17). □

Only a positive constant times $X/\log X$ is needed later. The explicit constant $1/2$ keeps the finite bounds transparent; optimizing it would not change the principal growth rate.

6 A first complete growth proof

Before sharpening the construction, it is useful to see why the base of the double exponential is already forced to be two.

Lemma 6.1 (Binary expansion bound). *For every integer $t \geq 1$, every positive integer $m < 2^{2^t}$ has an expression of height at most $2t + 1$.*

Proof. Expand m in binary. It is a sum of at most 2^t powers 2^e with $0 \leq e < 2^t$. The power $1 = 2^0$ has height zero, and all the other powers have height at most $t + 1$ by (12). A balanced sum of at most 2^t positive terms adds at most t levels. Zero binary digits are omitted; zero is not used as an expression leaf. $\square$

Fix $n \geq 16$, put $t = \lceil \log_2(2n) \rceil$, and let $k = 2t + 1$. All primes at most $X = 2^{2^n}$ are strictly below 2^{2^t} , even when $2^t = 2n$, because the upper endpoint X is composite. They therefore belong to S_k . Also $k \leq n$, and

$$P = \pi(X) \geq \frac{2^{2^n}}{4n \log 2}, \quad R = 2^{n-k}.$$

Since $2^k \geq 8n^2$, one has $P/R \geq 2^n$. Lemma 4.1 now gives

$$\log_2 a_n \geq R \log_2(P/R) \geq nR.$$

Finally $2^t < 4n$ implies $2^k = 2(2^t)^2 < 32n^2$, giving

$$\boxed{a_n \geq 2^{2^n/(32n)} \quad (n \geq 16).} \quad (18)$$

For completeness, $k \leq n$ follows from $k \leq 2 \log_2 n + 5 \leq n$ for $n \geq 16$. Combining (18) with (11) already proves

$$n - \log_2 n - 5 \leq \log_2 \log_2 a_n \leq n - 1,$$

and hence (5). The next section improves the logarithmic error to a square root of a logarithm. That improvement will also make a one-step growth theorem possible.

7 A sharper construction for arbitrary integers

Define the uniform height requirement

$$\mathcal{D}(t) = \max_{1 \leq m < 2^{2^t}} h(m) \quad (t \geq 0).$$

The interval is finite and nonempty, and $\mathcal{D}(0) = 0$. Define the triangular index

$$r(t) = \min\{r \geq 0 : t \leq r(r+1)/2\} = \left\lceil \frac{\sqrt{8t+1} - 1}{2} \right\rceil. \quad (19)$$

Theorem 7.1 (Uniform low-depth construction). *For every integer $t \geq 0$,*

$$\mathcal{D}(t) \leq t + r(t) + 2. \quad (20)$$

In particular, every positive integer with at most 2^t binary digits has such a representation using only 1, addition, and multiplication.

Proof. The case $t = 0$ is immediate. Induct on $t \geq 1$ and put $r = r(t)$. Minimality of r gives

$$\frac{(r-1)r}{2} < t \leq \frac{r(r+1)}{2}.$$

In particular $1 \leq r \leq t$. Set $q = t - r$. Then

$$0 \leq q \leq \frac{(r-1)r}{2}, \quad r(q) \leq r-1. \quad (21)$$

Split the binary digits of $m < 2^{2^t}$ into 2^r blocks, each of width $w = 2^q$ bits. Equivalently,

$$m = \sum_{j=0}^{2^r-1} d_j 2^{jw}, \quad 0 \leq d_j < 2^w. \quad (22)$$

For every nonzero digit d_j , the induction hypothesis and (21) give

$$h(d_j) \leq \mathcal{D}(q) \leq q + r(q) + 2 \leq (t-r) + (r-1) + 2 = t+1.$$

If $j > 0$, then $1 \leq jw < 2^t$, so $h(2^{jw}) \leq t+1$ by (12). The product $d_j 2^{jw}$ consequently has height at most $t+2$. The $j=0$ term needs no multiplication, but the same upper bound applies.

Discard zero terms in (22); at least one term remains. Their number is at most 2^r , so a balanced sum adds at most r levels. The complete expression has height at most $t+r+2$, as required. $\square$

The particular triangular numbers are not an unexplained guess. The construction uses r levels to add 2^r blocks. It is viable whenever the smaller coefficient problem at parameter $t-r$ has an overhead one unit below the current overhead. The recurrence

$$T_r = T_{r-1} + r, \quad T_0 = 0,$$

for the largest permitted parameter gives exactly $T_r = r(r+1)/2$.

7.1 What the construction proves about individual integers

For $m \geq 2$, take $t = \lceil \log_2(\log_2(m+1)) \rceil$, or equivalently choose the least t for which $m < 2^{2^t}$. The maximum-value obstruction and Theorem 7.1 give

$$h(m) = \log_2 \log_2 m + O(\sqrt{\log \log m}) \quad (m \rightarrow \infty), \quad (23)$$

and in particular $h(m)/\log_2 \log_2 m \rightarrow 1$. Here additive bounded errors caused by rounding or changing m to $m+1$ are absorbed in the stated error. The lower bound is $h(m) \geq 1 + \log_2 \log_2 m$ from (11).

This conclusion concerns depth, not expression size. Reusing powers of two in a DAG makes a convenient certificate, but reuse is not needed for the mathematical result: unfold the DAG into a tree, preserving height. Every leaf of the unfolded expression is still 1. No large constant is introduced for free.

7.2 Executable realization

The class `ExpressionBuilder` implements the proof directly. It constructs the nonzero block coefficients recursively, builds powers of two by balanced products of copies of $1 + 1$, multiplies the terms, and adds them in a balanced tree. A node table records only three operations: `one`, `add`, and `mul`.

The supplied independent checker evaluates the node table in topological order and recomputes every node height. The included 256-bit witness has 165 serialized nodes and height 14; its parameter is $t = 8$, for which the proved bound is $8 + 4 + 2 = 14$. Tests also exercise arbitrary-looking, all-ones, and single-bit inputs up to 16,384 bits. These are implementation checks of a construction already proved for all inputs, not empirical substitutes for Theorem 7.1.

8 Proof of the explicit counting bound

We now prove the first part of Theorem 1.1. Fix $n \geq 16$ and abbreviate

$$t = \lceil \log_2(2n) \rceil, \quad r = r(t), \quad k = t + r + 2, \quad R = 2^{n-k}.$$

The inequality $r \leq t$ gives

$$k \leq 2t + 2 \leq 2\log_2 n + 6 \leq n \quad (n \geq 16),$$

so R is a positive integer power of two.

Take $X = 2^{2n}$. As in Section 6, every prime at most X is strictly below 2^{2^t} . Theorem 7.1 puts all these primes in S_k , and Lemma 5.1 supplies

$$P \geq \frac{2^{2n}}{4n \log 2}.$$

Since $2^k \geq 2^{t+2} \geq 8n$, we have

$$\frac{P}{R} \geq \frac{2^{n+k}}{4n \log 2} \geq 2^n.$$

Prime-multiset amplification therefore yields

$$\log_2 a_n \geq R \log_2(P/R) \geq nR = n2^{n-k}. \quad (24)$$

Using $2^t < 4n$ in the final expression gives

$$n2^{n-k} = \frac{n2^n}{2^t 2^{r+2}} > 2^{n-r-4}.$$

Weakening the strict inequality proves the lower half of (3). The upper half is Proposition 3.3.

Finally, (19) gives

$$r_n = \sqrt{2 \log_2 n} + O(1).$$

Taking $\log_2$ twice in (3) proves (4). Since $\sqrt{\log n} = o(\log n)$, taking the natural logarithm once gives (6). This establishes the asserted global growth without any sum-product conjecture or prime-distribution hypothesis.

n	t_n	r_n	k	lower bound for $\log_2 \log_2 a_n$	upper bound
16	5	3	10	9	15
32	6	3	11	25	31
64	7	4	13	56	63
128	8	4	14	120	127
256	9	4	15	248	255
1024	11	5	18	1015	1023
10000	15	5	22	9991	9999
1000000	21	6	29	999990	999999

These rows evaluate the theorem, not the enormous sequence terms. For example, the row $n = 16$ asserts $a_{16} \geq 2^{512}$, not that a_{16} has been enumerated.

9 One-step growth through a divisor bound

The global estimate alone does not force $\log a_{n+1}/\log a_n \rightarrow 2$: broad upper and lower bounds do not rule out irregular local behavior. We control that behavior separately by bounding multiplicative collisions.

Let $\tau(z)$ denote the number of positive divisors of z .

Lemma 9.1 (Uniform divisor upper bound). *As $N \rightarrow \infty$,*

$$\log \max_{1 \leq z \leq N} \tau(z) \leq (\log 2 + o(1)) \frac{\log N}{\log \log N}. \quad (25)$$

Proof. Put $L = \log N$ and $y = L/(\log L)^3$; take N sufficiently large that $y > 1$. Factor $z = \prod p^{e_p}$. For primes $p \leq y$, there are at most y possible primes, and each exponent is at most $L/\log 2$. Their total contribution to $\log \tau(z) = \sum_p \log(e_p + 1)$ is at most

$$y \log(1 + L/\log 2) = O\left(\frac{L}{(\log L)^2}\right).$$

For primes $p > y$, the inequality $e + 1 \leq 2^e$ gives

$$\sum_{p > y} \log(e_p + 1) \leq (\log 2) \sum_{p > y} e_p \leq \frac{(\log 2)L}{\log y}.$$

The last step uses $\sum e_p \log p = \log z \leq L$. Since $\log y = \log L - 3 \log \log L$, this last expression is $(\log 2 + o(1))L/\log L$. Both bounds are uniform in $z \leq N$, proving (25). $\square$

Proposition 9.2. *Writing $b_n = \log a_n$, one has*

$$2b_n - (\log 2 + o(1)) \frac{2^n}{n} \leq b_{n+1} \leq 2b_n. \quad (26)$$

Proof. Consider all a_n^2 ordered pairs in $S_n \times S_n$ and map (x, y) to xy . For a fixed product z , at most $\tau(z)$ ordered pairs of positive integers can yield z , since the first operand is a divisor and determines the second. All these products are at most

$$M_n^2 = M_{n+1} = 2^{2^n}.$$

Hence, with $D_n = \max_{z \leq 2^{2^n}} \tau(z)$,

$$a_{n+1} \geq |S_n S_n| \geq \frac{a_n^2}{D_n}.$$

Apply Lemma 9.1 at $N = 2^{2^n}$. Since $\log N = (\log 2)2^n$ and $\log \log N = n \log 2 + O(1)$,

$$\log D_n \leq (\log 2 + o(1)) \frac{2^n}{n}.$$

Taking logarithms proves the lower bound. The upper bound is (13). $\square$

To finish Theorem 1.1, define $\varepsilon_n = 2 - b_{n+1}/b_n$ for $n \geq 1$. Proposition 9.2 and (3) give

$$\begin{aligned} 0 \leq \varepsilon_n &\leq \frac{(\log 2 + o(1))2^n}{n b_n} \\ &\leq \frac{(16 + o(1))2^{r_n}}{n} = n^{-1+o(1)}. \end{aligned}$$

Here $2^{r_n} = \exp(O(\sqrt{\log n})) = n^{o(1)}$. This proves (7), with all collision losses accounted for.

10 The precise computational cost

10.1 Candidate counts and unit-cost arithmetic

If $m = a_{j-1}$, one step has $m(m+1)/2$ unordered pairs and two mathematical candidate values per pair. Thus the exact candidate count in a cold run through stage n is

$$Q_n = \sum_{j=1}^n a_{j-1}(a_{j-1} + 1). \quad (27)$$

This counts evaluated sum/product candidates, not distinct outputs. The original program runs the pair iterator twice; the one-pass variant runs it once and computes both outputs inside the loop. The internal identity operations used by the built-in `sum` and `prod` only affect constant factors in this analysis.

By Proposition 3.4, successive cardinalities at least double. For $m = a_{n-1}$,

$$m(m+1) \leq Q_n \leq \frac{4}{3}m^2 + 2m,$$

so $Q_n = \Theta(a_{n-1}^2)$. In a unit-cost arithmetic model with bounded amortized set-operation cost, this is also the running-time order. The assertion about Q_n itself is deterministic and does not need a hash-table assumption.

For $n = 7$, the input size $a_6 = 67144$ gives

$$\binom{67145}{2} = 2\,254\,191\,940$$

unordered pairs, or 4 508 383 880 candidate values. The output is reported to contain 357 306 081 distinct integers. This jump explains why verifying through $n = 6$ is modest while casually rerunning the next stage is not. We did not run that next stage.

10.2 Bit operations and memory

The largest integer in S_n has

$$B_n = 2^{n-1} + 1 \quad (n \geq 1)$$

binary digits. Treating its arithmetic as permanently constant-time would therefore be inappropriate for an asymptotic bit-complexity claim. Let $M(B)$ be an upper bound for multiplying two B -bit integers. Under the usual bounded-average-probe hash-table model, an upper bound for a complete run is

$$O(a_{n-1}^2 (M(B_{n-1}) + B_n)). \quad (28)$$

The B_n term covers additions, hashing, and integer comparisons per bounded number of probes. This is a stated computational model, not a worst-case guarantee about every hash pattern in a particular interpreter. Hash collisions affect time but not the mathematical equality semantics of the set.

Even with every stage cached,

$$\sum_{j=0}^n a_j < 2a_n.$$

The old stage, the new stage, and the original program's temporary sum, product, and union sets therefore require $\Theta(a_n)$ stored entries up to fixed multiplicative factors, not $\Theta(na_n)$. A bit-storage upper bound is $O(a_n 2^n)$. An explicit, uncompressed collection of a_n distinct positive integers requires $\Omega(a_n \log a_n)$ bits: at least half of them are at least $a_n/2$. The ratio between these two bounds is only $n^{o(1)}$ by (6). Object headers and table slack affect practical memory constants, which are not estimated here.

10.3 An output-sensitive consequence

The local growth theorem implies

$$\frac{2 \log a_{n-1}}{\log a_n} \longrightarrow 1.$$

Therefore the candidate count satisfies

$$Q_n = a_n^{1+o(1)}. \quad (29)$$

This is logarithmic-scale near-linearity in the number of output integers, not near-linearity in n . The computation remains doubly exponential in n .

Using even schoolbook multiplication, the arithmetic factors in (28) are $2^{O(n)}$; their logarithms are negligible compared with $\log a_n$. Thus the same $a_n^{1+o(1)}$ conclusion holds for bit time under the stated hash-table model. A deterministic alternative is to sort and deduplicate the integer candidates. Its sorting factor and integer-comparison cost are also $a_n^{o(1)}$, so it attains the same logarithmic output exponent without a hashing assumption, though its memory layout and constants differ. No implementation of that large-scale sorting variant is claimed in the accompanying package.

11 Recomputed data and verification

The executable tests compare the one-pass and cached enumerators, a separate two-comprehension implementation, ordered Cartesian-pair enumeration through $n = 5$, and the

value-indexed recurrence (10) for every $m \leq 2000$. These checks exercise distinct ways of organizing the computation.

Let L_n be the largest integer with $[1, L_n] \subseteq S_n$, and let P_n be the number of primes in S_n . The following data were computed in the supplied run. A dash means that a quantity was not recomputed.

n	a_n	L_n	P_n	$\beta_n = 2^{-n} \log_2 a_n$
0	1	1	0	0.0000000000
1	2	2	1	0.5000000000
2	4	4	2	0.5000000000
3	11	9	4	0.4324289523
4	52	25	9	0.3562774824
5	678	177	56	0.2939106707
6	67144	7390	2033	0.2505464198
7	357306081*	—	—	0.2219733221*

* Reported OEIS term; not independently enumerated here. The small initial gaps are already visible at $n = 3$:

$$S_3 = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 12, 16\}.$$

For S_6 , the sumset has 21676 elements, the product set has 55883, and their intersection has 10415, giving

$$21676 + 55883 - 10415 = 67144.$$

These values refer to the sumset and product set formed *from* S_5 .

The prime counts do not require sieving all the way to M_n . A prime $p \in S_n$ has a minimum-height expression whose root cannot be a proper product. Its root is a sum of two elements of S_{n-1} , so $p \leq 2M_{n-1}$. For $n = 6$ this is 131072, despite $M_6 = 4294967296$. The data generator sieves only this smaller interval.

All 16 tests in the final test suite pass. They include input validation before deduplication, immutable-cache behavior, exact maxima, nesting, cardinality inequalities, power-of-two depths, the triangular recurrence, and independently evaluated expression certificates. The supplied run used Python 3.13.5; the implementation targets Python 3.9 or later and has no third-party dependencies. The runtime and platform are recorded in `verification/metadata.json`. The tests are finite validation; the all- n conclusions come from the proofs above.

12 What is established, and what remains to refine

The central established result is not merely that a_n is “very large.” Its double logarithm has slope exactly one in base two, with an explicit one-sided error of order $\sqrt{\log n}$; successive ordinary logarithms have ratio tending to two. The proof is constructive and every number-theoretic estimate used in it has been proved in this article.

There is also an exact way to express the remaining normalized constant. Define

$$d_n = \log_2 \left(\frac{a_n^2}{a_{n+1}} \right) \geq 0.$$

Then

$$\beta_n - \beta_{n+1} = \frac{d_n}{2^{n+1}}, \quad \theta = \frac{1}{2} - \sum_{n=1}^{\infty} \frac{d_n}{2^{n+1}}.$$

The series converges because its partial sums telescope and are bounded above by $1/2$. Its terms measure logarithmic collision loss relative to perfect squaring.

The present estimates do not decide whether that series exhausts $1/2$. If $\theta > 0$, then $\log a_n \sim (\theta \log 2)2^n$. If $\theta = 0$, the proved lower bound still forces an unusually slow vanishing: for every $\delta > 0$, eventually $2^{-n} \log a_n \geq n^{-\delta}$. Both possibilities are consistent with the results proved here. The first eight terms alone cannot choose between them, and their normalized logarithms are upper bounds for θ , not estimates with a certified error bar.

A route to a finer result would require tighter information about the multiplicative collision fibers of the actual sets S_n , or a more efficient source of multiplicatively independent constructible integers. The uniform divisor bound controls all integers up to M_{n+1} , including exceptional integers that need not be representative of these sets. Improving that specific overestimate is a concrete mathematical issue; it is not addressed by additional memoization or by counting expression syntax.

A Contents and reproduction instructions

The archive includes the following components.

article.tex, article.pdf

The complete mathematical article and its typeset version.

code/a352969.py

Validated cached and iterative enumerators, a proper-decomposition depth oracle, exact bound-parameter routines, the constructive expression builder, and the independent certificate checker.

code/test_a352969.py, code/reproduce.py

The regression tests and data-generation script.

data/

Statistics in CSV, all sets $S_0, \dots, S_6$ as compressed JSON, explicit-bound examples, and the expression witness.

verification/

Test output, data-generation output, and execution metadata.

From the archive root, reproduce the data with

```
python code/reproduce.py
python -m unittest discover -s code -p "test_*.py" -v
```

To compile the article, run `python build.py`, or use

```
latexmk -pdf -interaction=nonstopmode -halt-on-error article.tex
```

The default reproduction stops at $n = 6$. The general enumerator accepts a `pair_budget` keyword so a caller can reject an unexpectedly large stage before allocating its output. For example:

```
from a352969 import reachable, counting_bound_parameters

assert len(reachable(6, pair_budget=1_000_000)) == 67144
print(counting_bound_parameters(1_000_000))
```

The parameter routine uses exact integer arithmetic and does not try to allocate 2^{2^n} . Its returned exponents specify a theorem-backed bound, not a numerical enumeration.

B Source provenance

The OEIS page was retrieved on September 20, 2026. Its text-format record identifies revision 30, dated April 26, 2022. The mathematical definition, the listed terms, the program attribution, and the wrapper-name typo are source facts. The correctness proof, constructions, asymptotic estimates, complexity analysis, and executable package were developed for this note. No claim of priority over all prior mathematical literature is made. The original OEIS program is described rather than redistributed verbatim; the accompanying implementation is independently written.

References

- [1] V. Reshetnikov, *A352969*, The On-Line Encyclopedia of Integer Sequences. Python program credited to C. W. Wu, April 15, 2022. <https://oeis.org/A352969>. Text-format record: <https://oeis.org/search?q=id:A352969&fmt=text>. Accessed September 20, 2026.
- [2] Python Software Foundation, *itertools—Functions creating iterators for efficient looping*, documentation of `combinations_with_replacement`. https://docs.python.org/3/library/itertools.html#itertools.combinations_with_replacement. Accessed September 20, 2026.
- [3] Python Software Foundation, *functools—Higher-order functions and operations on callable objects*, documentation of `lru_cache`. https://docs.python.org/3/library/functools.html#functools.lru_cache. Accessed September 20, 2026.
- [4] Python Software Foundation, *Built-in Types*, integer arithmetic and set/frozenset semantics. <https://docs.python.org/3/library/stdtypes.html>. Accessed September 20, 2026.
- [5] Python Software Foundation, *math—Mathematical functions*, documentation of `prod`. <https://docs.python.org/3/library/math.html#math.prod>. Accessed September 20, 2026.
- [6] The OEIS Foundation, *A005245*, integer complexity measured by the minimum number of ones. <https://oeis.org/A005245>. Accessed September 20, 2026.
- [7] The OEIS Foundation, *A210659*, depth among representations having minimum operand complexity. <https://oeis.org/A210659>. Accessed September 20, 2026.